COMPLETE PROJECT APPROACH

PHASE 1 — PLAN THE PRODUCT FIRST

Before writing code, define:

1. What exactly does the app do?

Your app flow should be:

User signs up
→ uploads resume PDF
→ selects target role
→ backend extracts text
→ backend sends structured prompt to Gemini
→ Gemini returns analysis
→ backend structures response
→ save in database
→ frontend displays clean report

That is your ENTIRE product.

Do not add unnecessary features initially.

PHASE 2 — DEFINE TECH ARCHITECTURE

Frontend

Use:

- React
- Tailwind CSS
- Axios
- React Router

Backend

Use:

- Django
- Django REST Framework
- JWT Authentication

Database

Use:

- MySQL

AI

Use:

- Gemini API

PDF Processing

Use:

- pdfplumber (better extraction quality than PyPDF2)

PHASE 3 — DESIGN DATABASE STRUCTURE

Do this BEFORE coding.

TABLE 1 — USERS

id
name
email
password
created_at

TABLE 2 — RESUMES

id
user_id
resume_file
extracted_text
target_role
uploaded_at

TABLE 3 — ANALYSIS REPORTS

id
resume_id
ats_score
missing_skills

strengths
weaknesses
suggestions
full_ai_response
created_at

This already makes the project look much more professional than most student projects.

PHASE 4 — BUILD BACKEND FIRST

This is critical.

Most beginners waste time making beautiful UI first.

Wrong approach.

Your backend should work completely before frontend starts.

BACKEND DEVELOPMENT ORDER

STEP 1 — Django Setup

Create project:

django-admin startproject backend

Create apps:

python manage.py startapp users
python manage.py startapp resumes
python manage.py startapp analysis

STEP 2 — Install Dependencies

Use:

pip install django
pip install djangorestframework
pip install djangorestframework-simplejwt
pip install mysqlclient
pip install pdfplumber
pip install google-generativeai
pip install python-dotenv

STEP 3 — Authentication APIs

Build:

- register API
- login API
- JWT token system

Learn:

- serializers
- views
- URL routing
- token authentication

This phase teaches real backend development.

STEP 4 — Resume Upload API

Create API:

POST /upload-resume/

Flow:

receive PDF
→ save file
→ extract text
→ return extracted text

Use:

pdfplumber.open()

STEP 5 — TEXT EXTRACTION

This is where your app becomes interesting.

Flow:

PDF
→ pages

→ text extraction
→ combine text
→ clean text

You may need:

- remove extra spaces
- remove broken line breaks
- clean formatting

Store extracted text in DB.

STEP 6 — GEMINI INTEGRATION

THIS IS THE HEART OF THE PROJECT.

Most students do this badly.

Bad approach:

"Analyze this resume"

That looks weak.

GOOD APPROACH → STRUCTURED PROMPTS

Your prompt should force structured output.

Example logic:

You are an ATS resume analyzer.

Analyze the following resume for a Python Developer role.

Return:
1. ATS score (1-100)
2. Missing technical skills
3. Resume strengths
4. Weak sections
5. Improvement suggestions
6. Final verdict

Resume:
{resume_text}

Now the response becomes product-like.

STEP 7 — STRUCTURE AI RESPONSE

Very important.

Do NOT directly show raw Gemini output.

Instead:

- parse response
- organize into sections
- save properly

Example:

```
{
 "ats_score": 74,
 "missing_skills": [
  "Docker",
  "REST APIs"
 ],
 "strengths": [
  "Good project section"
 ]
}
```

THIS makes recruiters think:
“this person understands backend systems.”

PHASE 5 — BUILD FRONTEND

Only after backend works.

FRONTEND PAGES

1. Login Page

Features:

- login
- signup

- JWT storage

---

2. Dashboard

Show:

- uploaded resumes
- analysis history
- ATS scores

---

3. Upload Page

Features:

- PDF upload
- role dropdown
- loading spinner

---

4. Analysis Result Page

THIS PAGE SELLS YOUR PROJECT.

Display:

- ATS score card
- skill gaps
- strengths
- charts
- suggestions

Use:

- cards
- progress bars
- charts

PHASE 6 — MAKE IT LOOK PROFESSIONAL

This phase separates average projects from strong ones.

ADD THESE FEATURES

1. Loading Animation

While AI analyzes.

---

2. Upload Progress

Looks professional.

---

3. Resume History

Shows saved reports.

---

4. Download Report PDF

Very strong feature.

---

5. Charts

Example:

- ATS score meter
- skill match graph

Use:

- Chart.js
- Recharts

PHASE 7 — GITHUB

Critical mistake students make:
they upload messy code.

Do this properly.

GITHUB STRUCTURE

frontend/
backend/
README.md

WRITE A GOOD README

Include:

- project overview
- tech stack
- screenshots
- installation steps
- features
- architecture

This matters more than students think.

Use GitHub Docs README Guide for structure ideas.

PHASE 8 — DEPLOYMENT

Very important.

A deployed project is FAR stronger than localhost-only projects.

DEPLOYMENT OPTIONS

Frontend

Use:

- Vercel

Backend

Use:

- Render
- OR Railway

Database

Use:

- MySQL cloud DB
- or Railway DB

WHAT THIS PROJECT ACTUALLY TEACHES YOU

This project secretly teaches:

- REST APIs
- authentication
- file uploads
- AI integration
- prompt engineering
- database relationships
- backend architecture
- frontend/backend communication
- deployment
- production thinking

That is why this is MUCH stronger than generic CRUD projects.

BIGGEST MISTAKES TO AVOID

1. Copying YouTube blindly

You will not learn architecture.

---

2. Making UI first

Wrong priority.

---

3. Raw AI output

Looks amateur.

---

4. Overengineering

Do NOT add:

- interview bot
- voice AI
- LinkedIn scraper
- 50 features

Finish core product first.

RECOMMENDED BUILD TIMELINE

Week 1

Backend basics

- auth
- upload
- DB models

Week 2

Gemini integration

- text extraction
- structured analysis

Week 3

Frontend

- pages
- dashboard
- charts

Week 4

Polish

- deployment
- GitHub
- README
- testing